

PandOSsh — Guida allo studio per l'interrogazione

Struttura del progetto e del kernel

Nicolò Giuliani

Gaia Longo

Daniel Palamarciuc

A.A. 2025/2026

Indice

1	Visione d'insieme	1
2	Concetti della macchina μRISCV	1
3	Phase 1 — Strutture dati (Level 2)	2
3.1	PCB — Process Control Block	2
3.2	ASL — Active Semaphore List	2
4	Phase 2 — Il Nucleus (Level 3) [parte centrale]	3
4.1	Variabili globali del Nucleus	3
4.2	<code>initial.c</code> — Inizializzazione (boot del kernel)	3
4.3	<code>scheduler.c</code> — Scheduling	3
4.4	<code>exceptions.c</code> — Eccezioni e syscall	4
4.5	<code>interrupts.c</code> — Gestione degli interrupt	5
4.6	Il ciclo di una I/O (DIO) — da raccontare a voce	5
5	Phase 3 — Support Level (Level 4)	5
5.1	Memoria virtuale a paginazione	6
5.2	Swap Pool e Pager	6
5.3	U-proc e syscall utente	6
5.4	La shell interattiva	6
5.5	<code>crtsi.S</code> — startup delle U-proc (codice assembly)	7
6	Mappa della memoria	7
7	Domande tipiche d'esame	7

1 Visione d'insieme

PandOSsh è un sistema operativo didattico costruito *a strati* (layered) ed eseguito sull'emulatore μ RISCV (architettura RISC-V a 32 bit, `rv32imafd`). Ogni fase del progetto realizza un livello e poggia su quello sottostante:

Livello	Fase	Contenuto
1	(firmware/ROM)	BIOS dell'emulatore: bootstrap, gestione fisica eccezioni.
2	Phase 1	Strutture dati di base: PCB, code di processi, albero dei processi, ASL.
3	Phase 2	Nucleus : scheduler, gestione eccezioni/syscall, interrupt, device.
4	Phase 3	Support Level : memoria virtuale, U-proc in user-mode, syscall utente.

Idea chiave da ripetere all'esame: ogni livello fornisce un'*astrazione* al livello superiore e ne nasconde i dettagli. Phase 1 dà strutture dati pronte; Phase 2 le usa per costruire il concetto di *processo* e di *syscall*; Phase 3 usa le syscall del Nucleus per costruire *memoria virtuale* e *processi utente*.

Principio trasversale: nessuna allocazione dinamica. Tutto è preallocato in array statici e gestito con *free list*. Niente `malloc`: un OS deve essere *deterministico*, senza frammentazione né attese impreviste. Il numero massimo di processi (`MAXPROC=20`), semafori, frame, ecc. è noto a tempo di compilazione.

Build. Sistema **CMake**. I tre target (`phase1`, `phase2`, `phase3`) generano *tutti* lo stesso file `build/MultiPandOS.core.uriscv`, caricato dall'emulatore. Quindi va ricompilato il target giusto *prima* di lanciare la relativa configurazione (`config_machine.json` per fasi 1-2, `phase3_config_machine.json` per la 3).

2 Concetti della macchina μ RISCV

Vocabolario che l'esaminatore dà per scontato.

Kernel vs user mode

Il bit MPP nel registro `status` distingue *kernel-mode* (M, privilegiato) da *user-mode* (U). Le U-proc girano in user-mode; il Nucleus in kernel-mode.

Eccezione vs interrupt

Entrambe deviano il flusso, ma: un'**eccezione** è *sincrona* (causata dall'istruzione corrente: `syscall`, `page fault`, `trap`); un **interrupt** è *asincrono* (un device o un timer). Si distinguono dal **bit 31 del registro cause**: se acceso $\rightarrow$ interrupt; altrimenti il campo `ExcCode` indica il tipo di eccezione.

Pass Up Vector

Tabella (a `0x0FFFF900`) che dice al firmware *dove saltare* quando avviene un'eccezione: indirizzo dell'handler e puntatore allo stack da usare. Esistono due voci: una per il TLB-Refill, una per tutte le altre eccezioni.

BIOSDATAPAGE

Pagina (a `0x0FFFF000`) dove il firmware salva lo *stato del processore* al momento dell'eccezione. L'handler lo legge da qui (lo chiamiamo `savedState`).

SYSCALL / ECALL

Un processo richiede un servizio al kernel con l'istruzione `ECALL`, che genera un'eccezione (`ExcCode` 8 da U-mode, 11 da M-mode). Il numero della syscall sta in `a0`, gli argomenti in `a1`, `a2`, `a3`.

Semafori a contatore negativo

Un semaforo è un `int`. *P* (`PASSEREN`) decrementa: se diventa < 0 il processo si blocca. *V* (`VERHOGEN`) incrementa: se ≤ 0 sblocca un attendente. Il valore negativo conta quanti processi sono in attesa.

TOD / PLT / Interval Timer

TOD = orologio assoluto (per misurare il tempo CPU). PLT (Processor Local Timer) = timer per-CPU che genera l'interrupt di *fine time slice*. Interval Timer = timer di sistema che batte lo *pseudo-clock* ogni 100 ms.

3 Phase 1 — Strutture dati (Level 2)

Moduli `pcb.c` e `asl.c`. Sono i “mattoni”: non c'è ancora un kernel, solo strutture dati.

3.1 PCB — Process Control Block

Ogni processo è un `pcb_t`. Campi principali: stato del processore (`p_s`), tempo CPU usato (`p_time`), semaforo su cui è bloccato (`p_semAdd`), puntatore alla support struct (`p_supportStruct`), priorità (`p_prio`), PID (`p_pid`), e i link per le liste (`p_list`, `p_parent`, `p_child`, `p_sib`).

- **Free list:** i 20 PCB sono in un array statico; quelli liberi stanno in una lista. `allocPcb` ne prende uno (azzerandolo), `freePcb` lo restituisce. Allocazione/deallocazione $O(1)$.
- **Code di processi:** liste *doppiamente concatenate e circolari* (`list_head` stile kernel Linux). Inserimento/rimozione $O(1)$; niente casi speciali per coda vuota o con un elemento.
- **Albero dei processi:** ogni PCB ha un puntatore al padre e una lista dei figli (collegati tra loro come fratelli via `p_sib`). Permette di terminare un intero sotto-albero.

3.2 ASL — Active Semaphore List

Lista dei semafori *attivi* (con almeno un processo bloccato). Ogni descrittore `semd_t` contiene la chiave del semaforo e la coda dei PCB bloccati su di esso.

- Anche i `semd_t` usano una **free list** (numero massimo noto).
- La ASL è **ordinata per indirizzo** del semaforo: ricerca lineare ma con terminazione anticipata.
- `insertBlocked`: trova/alloca il descrittore e accoda il processo. `removeBlocked/outBlocked`: tolgono un processo; se la coda si svuota, il descrittore torna nella free list.

4 Phase 2 — Il Nucleus (Level 3) [parte centrale]

È il **kernel vero e proprio**. Quattro moduli: `initial.c` (boot), `scheduler.c` (scheduling), `exceptions.c` (eccezioni e syscall), `interrupts.c` (interrupt). Gira sempre in kernel-mode, con interrupt disabilitati nei punti critici, e *non* fa allocazioni dinamiche.

4.1 Variabili globali del Nucleus

Dichiarate in `initial.c`, condivise via `extern` (`globals.h`):

<code>processCount</code>	numero di processi vivi (creati e non ancora terminati).
<code>softBlockCount</code>	quanti processi sono bloccati in attesa di I/O o timer.
<code>readyQueue</code>	coda dei processi pronti a girare.
<code>currentProcess</code>	processo attualmente in esecuzione (NULL se nessuno).
<code>devSems[49]</code>	semafori dei device (48) + pseudo-clock (1).
<code>startTOD</code>	valore del TOD all'avvio della time slice corrente.
<code>activeProcs[20]</code>	array di <i>tutti</i> i processi vivi (vedi sotto).

4.2 initial.c — Inizializzazione (boot del kernel)

main() esegue, in ordine:

1. **Popola il Pass Up Vector:** handler eccezioni $\rightarrow$ `exceptionHandler`, handler TLB-Refill $\rightarrow$ `uTLB_RefillHandler`. *Scelta:* gli stack dei due handler sono i **due frame più alti della RAM** (`ramtop` e `ramtop-PAGESIZE`), non un unico `KERNELSTACK` — altrimenti due eccezioni annidate si sovrascrivono lo stack a vicenda.
2. Inizializza Phase 1 (`initPcbs`, `initASL`).
3. Azzerare le globali, inizializza i 49 semafori a 0.
4. *Legge il TOD reale* (`STCK`) in `startTOD` invece di azzerarlo: così il primo processo non accumula il tempo del bootstrap.
5. Avvia l'**Interval Timer** (`LDIT(PSECOND)`, 100 ms).
6. Crea *un* processo iniziale (`test`, priorità bassa), stack a `ramtop-2·PAGESIZE`, lo mette in ready queue.
7. Chiama lo `scheduler()` (che non ritorna mai).

4.3 scheduler.c — Scheduling

Politica: **round-robin con priorità**. Lo scheduler:

1. (gestione `YIELD`: vedi syscall `SYSS`) — se c'è un processo pronto di priorità $\geq$ a chi ha ceduto, quest'ultimo torna in coda.
2. Se la ready queue **non è vuota**: estrae il primo (`removeProcQ` rispetta la priorità), carica la **time slice** nel PLT (`setTIMER(TIMESLICE)`, 5 ms), salva il TOD e fa partire il processo con `LDST`.
3. Se `processCount == 0`: tutti i processi finiti $\rightarrow$ `HALT` (spegne la macchina).
4. Se `softBlockCount > 0`: ci sono processi in attesa di I/O. Il kernel entra in `WAIT` con tutti gli interrupt abilitati *tranne il PLT* (`MIE_ALL & ~MIE_MTIE_MASK`): non serve il timer mentre nessuno gira, e si evita un interrupt PLT spurio.
5. Altrimenti (vivi, nessuno pronto, nessuno soft-blocked): **deadlock** $\rightarrow$ `PANIC`.

4.4 exceptions.c — Eccezioni e syscall

Punto d'ingresso. `exceptionHandler` legge cause da `BIOSDATAPAGE` e smista:

- bit 31 acceso $\rightarrow$ `interruptHandler`;
- `ExcCode` 8/11 (`ECALL`) $\rightarrow$ `syscallHandler` (prima avanza il PC di una word, per non rieseguire la `ECALL`);
- `ExcCode` di TLB (page fault) $\rightarrow$ `tlbExceptionHandler` $\rightarrow$ `passUpOrDie(PGFAULTEXCEPT)`;
- tutto il resto $\rightarrow$ `programTrapHandler` $\rightarrow$ `passUpOrDie(GENERALEXCEPT)`.

Le syscall del Nucleus (numero in `a0`, negativo). Panoramica:

N.	Nome	Cosa fa
SYS1	CREATEPROCESS	Alloca un PCB figlio col dato stato/priorità/support, lo inserisce nell'albero e in <code>activeProcs</code> . Mette <i>padre e figlio</i> in ready queue e richiama lo scheduler (preemption se il figlio è più prioritario). Ritorna il PID in <code>a0</code> .

SYS2	TERMPROCESS	Termina un processo (e ricorsivamente tutti i discendenti). Se <code>a1==0</code> termina sé stesso, altrimenti cerca per PID.
SYS3	PASSEREN (P)	Decrementa il semaforo; se < 0 blocca il processo.
SYS4	VERHOGEN (V)	Incrementa il semaforo; se ≤ 0 sblocca il primo in attesa.
SYS5	DOIO	Avvia un'operazione di I/O su un device e <i>blocca sempre</i> il processo finché l'interrupt di completamento non lo risveglia (col device status in <code>a0</code>).
SYS6	GETTIME	Ritorna il tempo CPU accumulato dal processo.
SYS7	CLOCKWAIT	Blocca il processo fino al prossimo tick dello pseudo-clock (100 ms).
SYS8	GETSUPPORTPTR	Ritorna il puntatore alla support struct del processo.
SYS9	GETPROCESSID	Ritorna il PID del processo (o del padre se <code>a1≠0</code>).
SYS10	YIELD	Cede volontariamente la CPU (gestito via <code>yieldedProcess</code>).

Protezione. Una syscall *negativa* chiamata da *user-mode* genera un PRIVINSTR trap (le syscall del Nucleus sono solo per kernel-mode). Numeri ≥ 1 vengono passati su (sono syscall del Support Level).

Contabilità del tempo CPU (updateCPUTime). A ogni punto critico si calcola $\Delta = \text{TOD}_{\text{ora}} - \text{startTOD}$, lo si somma a `p_time`, e si aggiorna `startTOD`. Così nessun ciclo CPU va perso o contato due volte, anche con eccezioni annidate.

passUpOrDie (pass-up-or-die). Quando il Nucleus *non sa* gestire un'eccezione (page fault, program trap):

- se il processo ha una support struct $\rightarrow$ **pass up**: copia lo stato nell'area della support struct e salta all'handler di livello superiore (LDCXT);
- altrimenti $\rightarrow$ **die**: termina il processo e i suoi discendenti, poi richiama lo scheduler.

Questo **disaccoppia** il Nucleus dalla Phase 3: il kernel non conosce i gestori utente, li delega o uccide il processo.

Perché activeProcs? La ready queue contiene solo i processi *pronti*, non quelli bloccati. Ma TERMPROCESS per PID deve poter trovare anche un processo bloccato, e lo scheduler deve rilevare il deadlock. Quindi si tiene un array parallelo di *tutti* i processi vivi. È una **scelta aggiuntiva rispetto alla specifica** (costo $O(\text{MAXPROC})$, accettabile dato il limite fisso).

softBlockCount e isDeviceSemaphore. Il contatore conta solo i blocchi “soft” (attesa di un evento esterno: device o pseudo-clock), non i blocchi su semafori di sincronizzazione normali. Serve allo scheduler per decidere tra WAIT (qualcuno aspetta un interrupt) e PANIC (deadlock). `isDeviceSemaphore` distingue i due casi.

4.5 interrupts.c — Gestione degli interrupt

`interruptHandler` legge cause e distingue tre classi:

PLT (ExcCode 7) — fine time slice

Disarma il PLT (`setTIMER(NEVER)`), salva lo stato del processo corrente, lo rimette in ready queue (con interrupt riabilitati al riavvio) e richiama lo scheduler. È il meccanismo della **preemption**.

Interval Timer (ExcCode 3) — pseudo-clock

Riarma il timer (LDIT(PSECOND)), *sblocca in blocco* tutti i processi in attesa sul semaforo pseudo-clock (ognuno torna in ready queue con `a0=0`, `softBlockCount--`), azzerà quel semaforo, poi riprende il processo corrente o schedula.

Device (ExcCode 17–21) — linee 3–7

Legge la bitmap dei device pendenti, sceglie quello a priorità più alta (bit meno significativo). **Salva lo status prima di fare l'ACK** (altrimenti il valore andrebbe perso), poi fa la V sul semaforo del device sbloccando il processo in attesa (gli passa lo status in `a0`). Per i **terminali** (linea 7) TX e RX sono sotto-device separati, gestiti con priorità TX>RX per non perdere caratteri.

Al termine: se c'è un `currentProcess` lo riprende con LDST (niente context switch inutile), altrimenti chiama lo scheduler.

4.6 Il ciclo di una I/O (DOIIO) — da raccontare a voce

1. Il processo chiama DOIIO (SYS5) con l'indirizzo del registro di comando del device e il comando da scrivere.
2. Il Nucleus calcola *quale semaforo* corrisponde al device, scrive il comando nel registro, fa P sul semaforo e **blocca il processo**.
3. La macchina, non avendo processi pronti, va in WAIT.
4. Il device finisce, genera un **interrupt**. L'handler legge lo status, fa l'ACK, e V sul semaforo: il processo torna pronto col risultato in `a0`.

Niente race condition: il Nucleus gira con interrupt disabilitati tra l'emissione del comando e il blocco; l'interrupt resta pending finché non si va in WAIT.

5 Phase 3 — Support Level (Level 4)

Costruisce, *usando solo le syscall del Nucleus*, la **memoria virtuale** e i **processi utente** (U-proc) in user-mode. Tre moduli: `initProc.c`, `vmSupport.c`, `sysSupport.c`.

5.1 Memoria virtuale a paginazione

- Ogni U-proc gira nel segmento logico kuseg (da 0x80000000), con un **ASID** univoco $\in [1, 8]$.
- Ogni U-proc ha una **Page Table** privata di 32 entry: pagine 0–30 per `.text/.data`, entry 31 per lo *stack* (VPN 0xBFFFF).
- All'inizio tutte le pagine sono *non presenti* (V=0): il primo accesso genera un **page fault**.

5.2 Swap Pool e Pager

- Lo **Swap Pool** sono 16 frame fisici (= 2·UPROCMAX) a partire da 0x20020000 (dopo i 32 frame riservati all'OS). Una tabella (`swap_t`) dice cosa c'è in ogni frame (ASID, n. pagina, PTE).
- Un semaforo `swapPoolSem` (mutex) protegge la tabella: un solo page fault alla volta.
- Il **backing store** è il device flash: la U-proc con ASID *i* usa il flash *i* – 1. Le pagine vengono lette/scritte da lì.

Il Pager (TLB-Invalid handler). A ogni page fault: acquisisce `swapPoolSem`; sceglie un frame con algoritmo **FIFO** (round-robin); se occupato, *sfratta* la pagina vittima (invalida la sua PTE+TLB, la scrive sul suo flash); legge la pagina mancante dal flash nel frame; aggiorna Swap Pool table e Page Table; rilascia il mutex; riprende la U-proc.

TLB-Refill vs Pager — distinzione importante.

- Il **TLB-Refill** (`uTLB_RefillHandler`, in `exceptions.c`) è il *percorso veloce*: la pagina è già presente in Page Table ma manca solo nel TLB (cache). Copia l'entry nel TLB e riprende. Non tocca il disco.
- Il **Pager** gestisce il caso *vero* di pagina assente in memoria ($V=0$): deve andare sul flash. È costoso.

5.3 U-proc e syscall utente

- Le support struct sono in un array statico `supportPool[8]`, indicizzato per ASID.
- Syscall del Support Level (numero *positivo* in `a0`): `SYS2 TERMINATE`, `SYS4 WRITETERMINAL`, `SYS5 READTERMINAL`, `SYS6 EXECUTE`. Arrivano al `generalExceptionHandler` via `passUpOrDie`.

5.4 La shell interattiva

- L'`InstantiatorProcess` (`test`) avvia *una sola* U-proc: la **shell** (ASID 1). Le altre 5 (`echo`, `fibEleven`, `uname`, `sl`, `calc`) sono lanciate a runtime dalla shell con `SYS6` in base al comando digitato. La mappatura nome→ASID è statica in `shell.c`; ogni ASID i usa il device flash $i-1$ come backing store.
- **Catena di attesa** (due semafori): la shell si blocca su `shellSemaphore` finché la U-proc figlia non termina; l'`Instantiator` si blocca su `masterSemaphore` finché non termina la shell, poi `HALT`.
- **Terminazione ordinata** (`supTerminate`): prima di morire una U-proc libera i propri frame nello Swap Pool e fa la V sul semaforo giusto (master se è la shell, shell se è una figlia).

5.5 crtsh.S — startup delle U-proc (codice assembly)

- Ogni programma utente in `testers/` viene linkato con `crtsh.S` (più `liburiscv.S`): è il *C runtime startup*, l'equivalente per le U-proc del `crtso.S` usato dal kernel. Definisce il simbolo `__start`, posto dal linker a `0x800000B0` (`UPROCSTARTADDR`): è lì che la `SYS1` fa partire il PC, *non* su `main`.
- Fa il minimo indispensabile prima di `main`:
 1. carica il **global pointer** `gp` dal valore salvato nell'header `.aout` (accesso alle variabili globali);
 2. alloca un piccolo frame sullo stack (lo `sp` è già inizializzato dal kernel a `0xC0000000`) e chiama `jal main`;
- Se `main` ritorna, mette `a0 = 2` (`TERMINATE_SYSCALL`) ed esegue `ecall`: cioè una **SYS2** che fa terminare la U-proc in modo pulito. Senza questo, il ritorno da `main` salterebbe a un indirizzo casuale. (La shell non sfrutta questa via: ha un loop infinito e termina solo col comando `exit`.)
- *Da dire a voce*: `crtsh.S` è codice utente, gira in user-mode; il kernel garantisce solo PC e SP iniziali, tutto il resto del setup è qui.

5.6 Compilazione: da .c a device flash

- Ci sono **due build distinte**:
 1. Il **kernel** (core): `make phase3` (CMake) compila il Nucleus + Support Level e produce `MultiPandOS.core.uriscv` (header `.core`, linker script `uriscvcore.ldscript`). È il file caricato come `core-file` in `phase3_config_machine.json`.

2. Le **U-proc** di test: il **Makefile** in **testers/** produce un'immagine **flash .uriscv** per ogni programma. (Il target **phase3** richiama anche questo **make** per tenerle allineate.)
- Pipeline di una U-proc (per ogni **name.c**):
 1. **name.c** → **name.o** (compilazione, **-ffreestanding -nostdlib**);
 2. **link**: **name.o + crtisi.o + liburiscv.o** con il linker script **uriscvaout.ldscript** → **name.elf**. È qui che **crtisi.S** entra e **__start** finisce a **0x800000B0**;
 3. **uriscv-elf2uriscv -a** → **name.elf.aout.uriscv**: converte l'ELF nel formato **.aout** (load image) atteso dalla macchina;
 4. **uriscv-mkdev -f name.uriscv ...** → **immagine flash name.uriscv**: precarica il **.aout** su un device flash virtuale.
 - **Layout del flash** (backing store della U-proc): blocchi **0..30** = pagine **.text/.data**, blocco **31** = pagina di stack. Coincide con le 32 entry della Page Table.
 - **Mappatura** in **phase3_config_machine.json**: ogni device **flashN** punta a un **testers/*.uriscv**. La U-proc con ASID *i* legge il flash *i* - 1 (vedi **flashOperation: devNo = asid - 1**). Quindi **flash0=shell, flash1=echo, ...**
 - *Da dire a voce*: il flash **non** è un filesystem: è il backing store grezzo di *una* U-proc. Il Pager lo legge a domanda (page fault) e ci riscrive le pagine *dirty* sfrattate.

6 Mappa della memoria

Indirizzo	Costante	Contenuto
0x0FFFF000	BIOSDATAPAGE	stato salvato dal firmware sulle eccezioni
0x0FFFF900	PASSUPVECTOR	vettore degli handler (eccezioni + TLB-Refill)
0x10000000	—	registri dei device
0x20000000	RAMSTART	inizio RAM; primi 32 frame riservati all'OS
0x20020000	SWAP_POOL_START	16 frame dello Swap Pool
ramtop	—	stack handler eccezioni del Nucleus
0x80000000	KUSEG	inizio spazio logico delle U-proc
0x800000B0	UPROCSTARTADDR	ingresso .text di una U-proc
0xC0000000	USERSTACKTOP	cima dello stack utente

7 Domande tipiche d'esame

D: *Perché un OS non usa malloc?*

R: Per determinismo: niente frammentazione, niente attese impreviste, e un tetto noto alle risorse. Tutto è preallocato (array statici + free list, $O(1)$).

D: *Differenza tra eccezione e interrupt?*

R: L'eccezione è sincrona (l'istruzione corrente l'ha causata: syscall, page fault); l'interrupt è asincrono (device o timer). Si distinguono dal bit 31 del registro **cause**.

D: *Cos'è e come funziona la preemption qui?*

R: Allo scadere del time slice il PLT genera un interrupt (ExcCode 7); l'handler salva il processo, lo rimette in ready queue e schedula. Così nessun processo monopolizza la CPU.

D: *Cosa fa passUpOrDie?*

R: Quando il Nucleus non sa gestire un'eccezione: se il processo ha una support struct la passa al livello superiore (Phase 3); altrimenti termina il processo e i discendenti. Disaccoppia il kernel dai gestori utente.

D: *Perché serve `softBlockCount`?*

R: Per distinguere, quando la ready queue è vuota, il caso “qualcuno aspetta un interrupt” (→ WAIT) dal caso “deadlock” (→ PANIC). Conta solo i blocchi su device/pseudo-clock.

D: *Perché `DOIIO` blocca sempre il processo?*

R: Perché l’I/O è asincrona: si conclude con un interrupt di completamento. Il processo deve aspettare quel risultato. Non c’è race perché il Nucleus gira con interrupt disabilitati tra emissione comando e blocco.

D: *Differenza tra `TLB-Refill` e `Pager`?*

R: Il TLB-Refill ricarica nel TLB un’entry *già presente* in Page Table (veloce, niente disco). Il Pager gestisce una pagina *assente* in memoria: la legge dal flash, eventualmente sfrattando un’altra pagina (lento).

D: *Come si sceglie la pagina vittima?*

R: Algoritmo **FIFO** (round-robin sui 16 frame dello Swap Pool): semplice, $O(1)$, ammesso dalla specifica.

D: *Perché due semafori (*master* e *shell*) in *Phase 3*?*

R: Per la catena di attesa: una U-proc figlia sblocca la shell; la shell (al termine) sblocca l’Instantiator, che a quel punto fa HALT. Separarli rende esplicito chi aspetta chi.

D: *Perché lo stack degli handler è in cima alla RAM?*

R: Per isolare i due handler (eccezioni e TLB-Refill) su frame separati: con un unico stack, due eccezioni annidate si sovrascriverebbero a vicenda.

D: *Cosa succede a sistema “finito”?*

R: Quando `processCount` arriva a 0 lo scheduler chiama HALT: la shell termina → V su master → l’Instantiator fa `TERMPROCESS` su di sé → `processCount=0` → HALT.